Jordan Silver
CPSC – 350
5/20/23

This report presents the results of an empirical analysis comparing the execution times of various sorting algorithms, namely bubble sort, quick sort, merge sort, insertion sort, and selection sort. It explores the tradeoffs involved in selecting one algorithm over another, discusses the impact of using C++ as the programming language, and highlights the shortcomings of this empirical analysis.

Execution Time Comparison: The execution times of the sorting algorithms were measured and compared. The results indicated the following:

1. Bubble Sort: Bubble sort demonstrated relatively longer execution times compared to other algorithms, especially for larger datasets. Its time complexity of O(n^2) makes it less efficient, resulting in slower sorting.
2. Quick Sort: Quick sort performed well in terms of execution time, especially for large datasets. With an average time complexity of O(n log n), it exhibits good performance due to its efficient partitioning approach.
3. Merge Sort: Merge sort exhibited consistently fast execution times for all dataset sizes. Its time complexity of O(n log n) guarantees efficient performance by dividing the dataset into smaller, sorted sub lists before merging them.
4. Insertion Sort: Insertion sort showed acceptable execution times for small to moderately sized datasets. However, its time complexity of O(n^2) limits its scalability and makes it less suitable for larger datasets.
5. Selection Sort: Selection sort demonstrated relatively slower execution times compared to other algorithms. Its time complexity of O(n^2) and its sequential search for the minimum element in each iteration contribute to its inefficiency.

The selection of a sorting algorithm involves tradeoffs in terms of time complexity, space complexity, stability, and adaptability. Quick sort and merge sort offer superior time complexity with O(n log n), making them ideal for large datasets. However, quick sort is not stable, while merge sort requires additional memory for merging sublists. Bubble sort and insertion sort have simpler implementations but suffer from higher time complexity. Selection sort is simple to implement but lacks efficiency compared to other algorithms. Choosing C++ as the programming language for implementing the sorting algorithms had several effects on the results. C++ is a low-level language that provides

direct control over memory and efficient array manipulation. This allowed for optimized implementations of the sorting algorithms, resulting in faster execution times compared to higher-level languages.